they are not yet present on all nodes), which are used by the mapper or reducer program.

Hadoop pipes

With Hadoop pipes, you can implement applications that require higher performance in numerical calculations using C++ in MapReduce. The pipes utility works by establishing a persistent socket connection on a port with the Java pipes task on one end, and the external C++ process at the other.

Other dedicated alternatives and implementations are also available, such as Pydoop for Python, and *libhdfs* for C. These are mostly built as wrappers, and are JNI-based. It is, however, noticeable that MapReduce tasks are often a smaller component to a larger aspect of chaining, redirecting and recursing MapReduce jobs. This is usually done with the help of higher-level languages or APIs like Pig, Hive and Cascading, which can be used to express such data extraction and transformation problems.

Under the hood

We just saw how easily just a few lines of code can perform computations on a cluster, and distribute them in a manner that would take days for us to code manually. But now, let's just take a brief look at Hadoop MapReduce's job flow, and some background on how it all happens.

The story starts when the driver program submits the job configuration to the *JobTracker* node. As the name suggests, the *JobTracker* node controls the overall progress of the job, and resides on the *namenode* of the distributed filesystem, while monitoring individual success or failure of each Task. This *JobTracker* splits the *Job* into individual *Tasks* and submits them to respective *TaskTrackers*, which mostly reside on the *DataNodes* themselves, or at least on the same rack on which the data is present. This makes the HDFS filesystem rack-aware. Now, each *TaskTracker* receives its share of input data, and starts processing the map function specified by the configuration. When all the *Map Tasks* are completed, the *JobTracker* asks the *TaskTrackers* to start processing the reduce function. A *JobTracker* deals with failed and unresponsive tasks by running backup tasks, i.e., multiple copies of the same task at different nodes. Whichever node completes first, gets its output accepted. When both the *Map* and *Reduce* tasks are completed, the *JobTracker* notifies the client program, and dumps the output into the specified output directory.

The job flow, as a result, is made up of four main components:

- The Driver Program
- The JobTracker
- The TaskTracker
- The Distributed Filesystem

Limitations

I have been trumpeting the goodness of MapReduce for quite a bit of time, so it becomes essential to spend some time criticising it. MapReduce, apart from being touted as the best thing to happen to cloud computing, has also been criticised a lot:

- Data-transfer bandwidths between different nodes put a cap on the amount of data that can be passed around in the cluster.
- Not all types of problems can be handled by MapReduce, since a fundamental requirement for a problem is internal independence of data, and we cannot exactly determine or control the number or order of *Map* and *Reduce* tasks.
- A major (the most important) problem in this paradigm is the barrier between the *Map* phase and *Reduce* phase, since processing cannot go into the *Reduce* phase until all the map tasks have completed.

Last-minute pointers

Before leaving you with your own thoughts, here are a few pointers:

- The HDFS filesystem can also be accessed using the Namenode's Web interface, at *http://localhost:50070/*.
- File data can also be streamed from *Datanodes* using HTTP, on port 50075.
- Job progress can be determined using the Map-reduce Web Administration console of the *JobTracker* node, which can be accessed at *http://localhost:50030/*.
- The *jps* command can be used to determine the ports at which Hadoop is active and listening.
- The *thrift* interface is also an alternative for binding and developing with languages other than Java. END

Further reading

[1] Apache Hadoop Official website - http://hadoop.apache.org/
[2] Disco Project - http://discoproject.org/
[3] Cloudera's free Hadoop Training lectures - http://bit.ly/btL6Ad
[4] Google's lectures and slides on Cluster Computing and Mapreduce - http://bit.ly/44YUv
[5] Google's MapReduce Research Paper - http://labs.google.com/papers/mapreduce.html
[6] MapReduce: A major step backwards - http://bit.ly/4ZRbWY

By: Ankit Mathur

The author is a 21-year old geek. He has a crush on Java, and also loves flirting with almost all other stuff related to Web technologies. Feel free to poke fun at this article, and direct your feedback to *ankitreloaded@gmail.com*.